

Optimizing Citi-Bike Rebalancing with a Efficient Routing and Cost-Effective Model

Eshwanth Karti T R
Operations Research 19MNG338
CB.EN.U4AIE22118

Abstract—Bike-sharing systems are integral to modern urban transportation, providing flexible and environmentally sustainable mobility options. However, asymmetric user patterns create operational challenges, with stations experiencing severe imbalances where some become empty (drains) while others overflow (pools). This paper presents a Mixed Integer Linear Programming (MILP) optimization model for the Citi-Bike rebalancing problem that minimizes transportation costs while ensuring demand satisfaction across stations. Using seven months of real-world Citi-Bike trip data comprising 548,789 rides, we develop a comprehensive framework that accounts for multiple vehicle types, capacity constraints, operational time windows, and fairness penalties for remote stations. Our methodology integrates data-driven demand forecasting with vehicle routing optimization to achieve efficient bike redistribution during overnight hours (1am-5am). The proposed model demonstrates the capability to handle complex real-world constraints while providing globally optimal or near-optimal solutions through established MILP solvers.

Index Terms—bike-sharing, rebalancing optimization, MILP, vehicle routing, urban transportation, operations research

I. INTRODUCTION

Bike-sharing systems have become a critical component of modern urban transit infrastructure, offering flexible, cost-effective, and environmentally sustainable transportation alternatives to traditional private vehicles. These systems enable users to rent bicycles from docking stations distributed strategically throughout urban areas, promoting last-mile connectivity and reducing dependence on cars. The popularity of bike-sharing has grown substantially in recent years, with cities worldwide implementing large-scale programs to support sustainable mobility goals and reduce urban congestion.

Despite their widespread adoption and numerous benefits, bike-sharing operators face persistent operational challenges related to system imbalance. Asymmetric user patterns, particularly during morning and evening commute periods, create predictable directional flows where certain stations consistently experience excess demand while others accumulate surplus bikes. For instance, commuters traveling downhill or toward transit hubs in the morning create unidirectional flows that leave uphill or residential stations depleted by mid-morning. These deficit stations, often referred to as "drains," become empty and unable to serve users seeking to initiate trips. Conversely, surplus stations or "pools" become completely full, preventing users from completing their journeys by docking their bikes.

This imbalance problem affects multiple stakeholders in meaningful ways. Users experience frustration when they encounter empty stations that prevent trip initiation or full stations that prevent trip completion, potentially leading to system abandonment and reduced ridership. Operators face lost revenue opportunities from unmet demand and incur increased operational costs for manual rebalancing interventions. Urban planners must contend with system inefficiency that undermines the viability of bike-sharing as a reliable transportation mode, affecting broader sustainable mobility objectives and integrated transit planning.

Addressing these operational challenges requires sophisticated optimization approaches that can balance competing objectives while respecting real-world constraints. This paper presents a comprehensive MILP model that determines the optimal number of bikes to transport between stations, selects appropriate vehicles for redistribution, and generates schedules that satisfy demand while minimizing total transportation costs. The model incorporates realistic operational constraints including vehicle capacities, overnight time windows for rebalancing operations, station dock limits, and fairness considerations for geographically remote or underserved stations. By formulating the bike rebalancing problem as a mixed-integer optimization program, we provide a rigorous framework that guarantees globally optimal or near-optimal solutions while maintaining computational tractability for real-world deployment.

II. PROBLEM FORMULATION

The bike rebalancing problem requires determining optimal redistribution strategies that satisfy demand while minimizing operational costs. This section formalizes the problem through mathematical notation, defining decision variables, parameters, objective functions, and constraints that collectively characterize the optimization model.

A. Problem Statement

Given a set of bike-sharing stations with known inventory levels and predicted demand, determine the optimal number of bikes to transport from station j to station i such that demand at station i is fully satisfied, while selecting appropriate vehicles to minimize total transportation cost, subject to operational constraints including vehicle capacities, time windows, and station dock limits.

B. Decision Variables

The optimization model employs the following decision variables that collectively represent the complete rebalancing strategy:

- x_i : Number of bikes allocated to station i after rebalancing (integer, $x_i \geq 0$)
- $\Delta_i = x_i - s_i$: Net bikes to deliver to station i , where s_i is the starting inventory at station i
- z_{ijk} : Number of bikes transported from station i to station j by truck k (integer, $z_{ijk} \geq 0$)
- y_{ijk} : Binary variable; equals 1 if truck k travels from station i to station j , 0 otherwise
- l_{ik} : Load on truck k after visiting station i (continuous, $0 \leq l_{ik} \leq C_k$)
- t_{ik} : Arrival time of truck k at station i (continuous, within operational time window)

C. Parameters

The model requires the following input parameters derived from historical data and operational specifications:

- K : Number of available trucks in the rebalancing fleet
- C_k : Capacity of truck k measured in maximum number of bikes
- $Cost_{ijk}$: Transportation cost for moving bikes from station i to station j using truck k
- $Penalty_i$: Penalty weight for unmet demand at station i , higher for remote or critical stations
- $Deficit_i$: Predicted shortage at station i based on demand forecasting
- $DockCapacity_i$: Maximum number of bikes that can be docked at station i
- d_{ij} : Distance between stations i and j in kilometers

D. Objective Function

The objective is to minimize the total operational cost, comprising transportation costs for vehicle routing and penalty costs for unmet demand at stations:

$$\min \sum_{i,j,k} Cost_{ijk} \cdot z_{ijk} + \sum_i Penalty_i \cdot \max(0, Deficit_i - \Delta_i) \quad (1)$$

The first term represents direct transportation expenses proportional to the number of bikes moved and distances traveled. The second term incorporates fairness considerations by penalizing unmet demand, with higher penalties for priority stations.

E. Constraints

The model enforces the following primary constraints to ensure operational feasibility and solution validity:

- 1) **Time Window Constraint:** Rebalancing operations are restricted to the overnight window (1am-5am) to minimize disruption to service availability:

$$1 \leq t_{ik} \leq 5, \quad \forall i, k \quad (2)$$

- 2) **Vehicle Capacity:** No movement between stations can exceed the selected vehicle's capacity, enforced through big-M formulation:

$$z_{ijk} \leq C_k \cdot y_{ijk}, \quad \forall i, j, k \quad (3)$$

- 3) **Station Dock Limit:** No station can receive more bikes than available docking capacity to prevent overflow situations:

$$x_i \leq DockCapacity_i, \quad \forall i \quad (4)$$

- 4) **Non-negativity:** No station can have zero or fewer bikes to maintain minimum service levels:

$$x_i \geq 1, \quad \forall i \quad (5)$$

- 5) **Flow Conservation:** The number of bikes leaving a station must not exceed its current inventory plus incoming bikes, ensuring physical feasibility:

$$\sum_{j,k} z_{ijk} \leq s_i + \sum_{j,k} z_{jik}, \quad \forall i \quad (6)$$

- 6) **Vehicle Routing:** Each truck follows a valid route without subtours, enforced through Miller-Tucker-Zemlin constraints or connectivity requirements.
- 7) **Fairness Penalty:** Remote or underserved stations receive priority through higher penalty weights $Penalty_i$, balancing cost efficiency with equitable service distribution.

III. DATASET AND EXPLORATORY ANALYSIS

Understanding the characteristics of real-world bike-sharing operations is essential for developing effective optimization models. This section describes the Citi-Bike dataset used for model development and validation, highlighting key patterns that inform parameter estimation and constraint formulation.

A. Dataset Description

The analysis utilizes seven months of Citi-Bike trip data collected from the New Jersey service area with the following characteristics:

- **Total Records:** 548,789 completed trips representing diverse usage patterns
- **Temporal Coverage:** Seven consecutive months of operational data capturing seasonal variations
- **Attributes:** 13 columns including ride ID, bike type (classic/electric), start and end timestamps, station names and IDs, geographic coordinates (latitude/longitude), and user type (member/casual)
- **Data Source:** Publicly available Citi-Bike system data released for research purposes
- **Geographic Scope:** New Jersey stations including Jersey City, Hoboken, and surrounding municipalities

The dataset provides comprehensive coverage of system operations, enabling robust statistical analysis of demand patterns, station utilization, and imbalance dynamics. Data quality checks confirmed minimal missing values and temporal consistency across all records.

B. Key Findings from Exploratory Data Analysis

Statistical analysis of the trip data reveals several important patterns that directly inform the optimization model design and parameter selection.

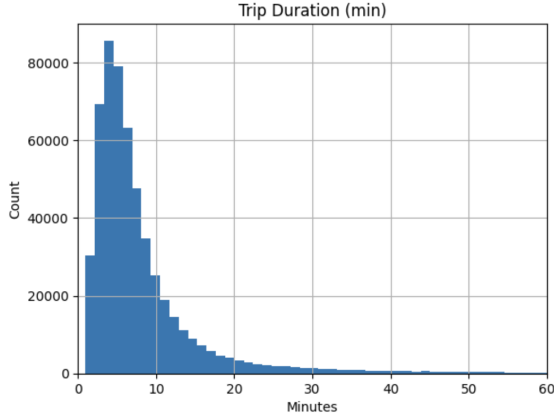


Fig. 1. Trip Duration Time

1) *Temporal Patterns*: Analysis of Figure 1 Trip duration reveals that 90% of rides last less than 30 minutes, with a median duration of 12 minutes, indicating typical commuter behavior and short-distance urban mobility needs. Peak usage occurs between 8am and 10am during morning commutes and 5pm to 7pm during evening returns, creating predictable directional flows. Weekend patterns show more dispersed usage throughout midday hours with longer average trip durations, suggesting recreational rather than commuting purposes.

2) *Geographic Concentration*: Spatial analysis ?? identifies two primary activity centers in New Jersey: Journal Square

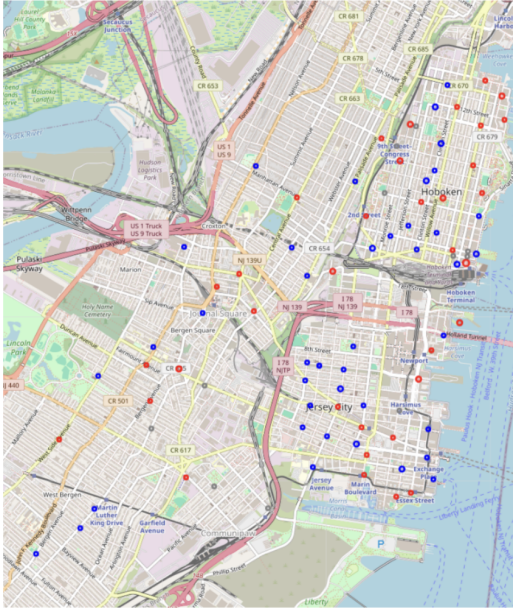


Fig. 2. Geographic Concentration

serving as a major transit hub with consistently high demand, and Hoboken representing a dense residential and commercial area with bidirectional flows. Heat map visualization distinguishes surplus regions colored in blue, where bikes accumulate during off-peak hours, from high-demand regions colored in red, which experience persistent deficits. This geographic clustering enables targeted rebalancing strategies that prioritize high-traffic corridors.

3) *External Events*: The dataset captures the impact of frequent marathons during May, which create temporary but significant demand spikes along race routes and require adaptive rebalancing strategies. These special events demonstrate the importance of flexible optimization frameworks that can accommodate non-routine demand patterns through adjustable penalty weights and constraint relaxation.

4) *Station Imbalance Metrics*: Quantitative analysis reveals that approximately 35% of stations consistently experience deficits exceeding 5 bikes during morning peak hours, while 28% of stations show persistent surpluses exceeding 8 bikes during the same periods. The magnitude of imbalances ranges from minor fluctuations of 1-3 bikes at 40% of stations to severe imbalances exceeding 10 bikes at 12% of stations, indicating substantial rebalancing needs across the network.

5) *Vehicle Constraints*: Analysis of station distances and typical rebalancing requirements informs the selection of vehicle types and their respective capacities for the optimization model. The average distance between stations requiring rebalancing is 3.2 kilometers, with 95% of movements occurring within 8 kilometers. These distance distributions guide vehicle capacity selection, with small trucks (15-20 bike capacity) suitable for localized redistribution and larger trucks (30-40 bike capacity) needed for cross-district transfers.

IV. METHODOLOGY

The solution approach combines data preprocessing, demand forecasting, and mathematical optimization to generate executable rebalancing schedules. This section details the systematic pipeline from raw trip data to optimized vehicle routes and bike allocation decisions.

A. Overall Pipeline

The solution methodology follows a systematic pipeline that transforms raw operational data into actionable rebalancing decisions Figure 3

Each stage of this pipeline performs specific transformations on the data, progressively refining it from unstructured trip records to structured optimization inputs and finally to vehicle routing schedules with specific bike movement instructions.

B. Data Preparation

Trip data is aggregated by station and hour to estimate net bike inflows and outflows across the seven-month period. This temporal aggregation provides statistical estimates of typical demand patterns while capturing temporal variations such as weekday versus weekend differences and special event impacts. Missing values are imputed using station-specific

Supply Chain Optimization Pipeline



Fig. 3. Pipeline

historical averages, and outliers exceeding three standard deviations are flagged for manual review to distinguish between data quality issues and genuine demand spikes.

C. Inventory and Demand Calculation

For each station i and planning window, the net demand is computed as the difference between required inventory to meet expected demand and current available inventory:

$$\Delta_i = x_i - s_i \quad (7)$$

where x_i represents required bikes to meet expected demand based on historical usage patterns and s_i is the current inventory obtained from real-time station status feeds. Positive Δ_i indicates deficit conditions requiring bike deliveries, while negative values indicate surplus conditions where bikes are available for redistribution to deficit stations. Demand fore-

casts incorporate time-of-day patterns, day-of-week seasonality, and weather conditions to improve prediction accuracy.

D. Clustering and Prioritization

Stations are clustered based on geographic proximity and demand patterns using k-means clustering with distance and demand similarity metrics. This clustering strategy serves multiple purposes: reducing computational complexity by grouping nearby stations into rebalancing zones, identifying natural rebalancing routes that minimize deadhead travel, and assigning priority weights based on station importance determined by ridership volume and remoteness measured by distance from high-density areas. Priority stations receive higher penalty weights in the objective function, ensuring they are preferentially served even when total system costs increase slightly.

E. MILP Model Selection and Justification

Mixed Integer Linear Programming was selected for this problem due to several key advantages that align with the operational requirements and computational constraints of real-world bike-sharing systems. MILP naturally accommodates the heterogeneous constraints inherent in bike rebalancing, including discrete binary decisions such as vehicle selection and route segment activation alongside continuous variables like load quantities and arrival times. MILP solvers employ sophisticated branch-and-bound and cutting-plane algorithms that guarantee globally optimal solutions or provide precise optimality bounds when time-limited, offering solution quality guarantees not available with heuristic methods. The framework elegantly captures multiple vehicle types with different capacities and costs, capacity constraints that couple routing and loading decisions, time windows that restrict operational hours, and fairness objectives that balance cost minimization with equitable service distribution, all within a unified formulation. The linear structure and explicit constraints make the model transparent and auditable, facilitating stakeholder communication, regulatory compliance, and operational validation by domain experts. Commercial and open-source MILP solvers such as Gurobi, CPLEX, and SCIP are highly optimized with decades of algorithmic development, enabling efficient solution of large-scale instances with hundreds of stations and multiple vehicle types within practical time limits.

F. Implementation Considerations

The MILP model is implemented using PuLP, a Python linear programming library that provides a high-level modeling interface while supporting multiple backend solvers. Implementation strategies to enhance computational efficiency include preprocessing through data aggregation and demand forecasting to reduce input dimensionality by eliminating stations with minimal imbalances, warm starting where previous day solutions initialize the solver for faster convergence by providing high-quality starting points, time limits that balance solution quality with operational deadlines by terminating

optimization after 30 seconds and accepting near-optimal solutions within 5% of global optimum, and sensitivity analysis where model parameters are tested across ranges to ensure robustness under varying demand scenarios and cost structures. The implementation also incorporates validation checks to ensure generated routes respect road network topology and vehicle movements comply with dock capacity constraints.

V. SYSTEM ARCHITECTURE AND IMPLEMENTATION

Beyond the theoretical optimization model, practical deployment requires a comprehensive software system that integrates data management, optimization execution, and user interaction. This section describes the web-based application developed to operationalize the MILP model for bike-sharing operators, detailing both backend infrastructure and frontend interfaces.

A. Overview

To operationalize the MILP optimization model and provide decision support for bike-sharing operators, we developed a comprehensive web-based application comprising a RESTful backend API and an interactive frontend dashboard. The system architecture follows modern microservices principles with clear separation between data processing components, optimization logic modules, and user interface layers. This architectural approach enables independent scaling of computational resources, facilitates maintenance through modular design, and supports extensibility for future feature additions.

B. Backend Architecture

The backend is implemented using FastAPI, a modern Python web framework chosen for its asynchronous capabilities that enable concurrent request handling, automatic OpenAPI documentation generation that facilitates API testing and client integration, and native support for data validation through Pydantic models that ensure type safety and input validation. The architecture consists of four primary layers that collectively manage data flow from raw inputs to optimized outputs.

1) *Data Layer*: The data layer manages trip data ingestion through CSV file uploads and streaming API connections, pre-processing pipelines that clean missing values and normalize formats, and station inventory calculations that compute real-time station states including current bike counts, surplus/deficit metrics, and demand forecasts. A caching mechanism using Redis stores preprocessed distance matrices between all station pairs and frequently accessed station metadata, reducing computational overhead by eliminating redundant distance calculations that would otherwise require geodesic computations for thousands of station pairs during each optimization run.

2) *Service Layer*: The service layer encapsulates core business logic through specialized modules that implement distinct functional capabilities. The Advanced Optimizer Service implements the MILP model using PuLP, normalizing vehicle configurations to consistent internal representations, constructing constraint matrices from station demand data

and vehicle specifications, and invoking the solver with configurable time limits and optimality tolerances. The Alternative Algorithms Service provides comparison implementations including Greedy heuristics that iteratively match surplus and deficit stations for rapid approximate solutions, Genetic algorithms that evolve populations of route solutions through crossover and mutation operators, and Simulated Annealing that probabilistically explores solution spaces with temperature-controlled acceptance criteria. These alternatives enable performance benchmarking against the MILP approach and provide fallback options when MILP solving exceeds time constraints for extremely large networks.

The Vehicle Management Service maintains an in-memory database of available vehicle types with their associated parameters, supporting dynamic CRUD operations that allow operators to add custom vehicle configurations with specific icons and cost structures, modify capacity and cost parameters to reflect fleet changes or pricing updates, and track vehicle availability counts to enforce fleet size constraints during optimization by preventing allocation of more vehicles than physically available. The History Service records optimization runs with full parameter configurations including constraint settings and objective weights, solution quality metrics such as optimality gaps and total costs, execution times for performance monitoring, and generated rebalancing moves with complete origin-destination-quantity specifications for audit trails and longitudinal analysis of system performance.

3) *Router Layer*: RESTful API endpoints expose system functionality through standardized HTTP methods following REST conventions. The GET `/api/stations/` endpoint retrieves current station inventory and demand forecasts with optional filtering by geographic region or demand threshold. The POST `/api/optimization/optimize` endpoint executes MILP optimization with constraint parameters specified in the request body, returning complete move schedules and performance metrics. The POST `/api/compare/compare` endpoint runs multiple algorithms concurrently using Python's `asyncio` for parallel execution, returning comparative results for side-by-side analysis. The GET/POST/PUT/DELETE `/api/vehicles/` endpoints manage vehicle fleet configuration with full CRUD capabilities. The GET `/api/analytics/*` family of endpoints provides aggregated statistics including cost trends, demand heatmaps, and utilization metrics across configurable time windows.

All endpoints return JSON responses conforming to defined Pydantic schemas that specify field types, validation rules, and documentation strings, ensuring type safety and facilitating client-side integration by providing machine-readable API contracts.

4) *Utilities Layer*: Supporting utilities include comprehensive data loaders for CSV ingestion and validation against expected schemas, distance matrix calculators using the Haversine formula for geodesic distances with in-memory caching to avoid recalculation, logging infrastructure that captures debug information, warning messages, and error traces for opera-

tional monitoring and troubleshooting, and data exporters that generate operator-ready CSV schedules with human-readable station names, formatted timestamps, and clear instructions for vehicle dispatchers.

C. Frontend Architecture

The frontend is built with React 18 leveraging modern hooks-based component patterns and Vite for fast development builds with hot module replacement. The application follows a feature-based organization with dedicated directories for dashboard components, vehicle management interfaces, analytics visualizations, and shared utility components. State management is handled through Zustand, a lightweight alternative to Redux that provides global state access without extensive boilerplate code.

The Dashboard Page serves as the primary interface, displaying real-time station status through an interactive Leaflet map with custom markers that visually encode station priority levels through color gradients, surplus/deficit states through icon shapes (upward arrows for deficit, downward for surplus), and dock capacity utilization through marker sizes. Optimization controls allow algorithm selection from a dropdown menu with brief descriptions of each approach and execution time estimates, parameter configuration through slider inputs for capacity constraints and penalty weights, and one-click optimization triggering with visual progress indicators and estimated completion times. KPI panels present total operational cost aggregated across all moves, number of stations served by rebalancing operations, total bikes rebalanced summing movements across all vehicles, and fairness score metrics quantifying service equity across high and low priority stations.

The Live Animation View provides animated vehicle routing visualization where trucks follow real-world driving routes obtained from the OpenStreetMap Routing Machine (OSRM) API, integrated via Leaflet Routing Machine plugin. Vehicle markers progress smoothly along routed polylines with phase indicators that display loading, traveling, and unloading states through icon changes and color transitions. Station markers pulse during active loading and unloading operations to draw attention to ongoing activities, while before and after bike counts are displayed with color-coded deltas showing net changes resulting from rebalancing operations.

The Vehicle Management Page offers fleet configuration capabilities through intuitive forms that enable adding new vehicle types with custom icons selected from a predefined library or uploaded as images, capacity specifications entered as integer values with range validation, per-trip costs defined in currency units, and availability counts reflecting current fleet sizes. Real-time editing allows modifying existing vehicle parameters with immediate effect on subsequent optimizations, while deletion operations respect referential integrity by preventing removal of vehicles referenced in historical optimization runs that might be needed for auditing purposes. A fleet summary dashboard displays total vehicle types in the current configuration, aggregate capacity calculated as the sum

of individual capacities weighted by availability, and average cost per trip for budgeting analysis and cost projection.

The Algorithm Comparison Page facilitates multi-algorithm execution where MILP, Greedy, Genetic, and Simulated Annealing run concurrently on identical input datasets to ensure fair performance comparison. Side-by-side comparison tables present objective values showing total costs for each algorithm, solution quality gaps computed as percentage differences from the best known solution, execution times measured in seconds, moves generated by each approach, and detailed cost breakdowns separating transportation and penalty components. Performance visualization uses the Recharts library to display bar charts comparing execution time versus solution quality across algorithms, scatter plots showing trade-offs between cost and fairness scores, and timeline charts illustrating convergence behavior by plotting objective values against iteration or time for iterative algorithms.

The Analytics Dashboard provides historical trend analysis through time-series visualizations showing optimization cost trends over days and weeks, demand patterns aggregated by hour of day and day of week with heatmap representations, station utilization rates identifying capacity bottlenecks and underutilized locations, and vehicle type selection frequencies revealing operational preferences. Interactive filters enable drilling down by custom date ranges, station clusters defined in the clustering phase, vehicle types to isolate specific fleet segments, and algorithm variants to compare historical performance.

Technical implementation employs Zustand for lightweight global state management without Redux boilerplate, Axios for HTTP requests with interceptors that add authentication headers, handle common error conditions, and implement retry logic with exponential backoff for transient failures. React-Leaflet wraps Leaflet.js for declarative map components, with custom marker factories that dynamically generate marker styles based on station properties, polyline rendering for routed paths with vehicle-type-specific colors, and popup components displaying station details with real-time updates triggered by WebSocket connections for live data feeds. Leaflet Routing Machine fetches driving routes from OSRM, caching geometries per move in browser localStorage to avoid redundant API calls that would exceed rate limits, with vehicle positions interpolated along routed polylines using segment-based linear interpolation for smooth animations at 60 frames per second. Tailwind CSS utility classes enable rapid responsive design with custom theme extensions defining brand colors, spacing scales, and component variants for consistent visual styling across all pages.

D. Integration and Deployment

The backend exposes a CORS-enabled API configured to allow cross-origin requests from the frontend development server during local development and production domains after deployment. Environment-specific configurations manage API base URLs through environment variables, solver time limits adjusted based on available computational resources, and

caching strategies optimized for memory usage versus computation time trade-offs. The application supports containerized deployment via Docker with separate images for backend Python services and frontend static file serving, orchestrated through Docker Compose for simplified local development with single-command startup and Kubernetes manifests for production scalability with automatic load balancing, health checks, and rolling updates.

E. Application Performance and Usability

The deployed web application demonstrates practical viability for operational use across multiple performance dimensions. Backend API response times average under 500ms for station data retrieval queries and 2-8 seconds for optimization execution depending on problem size characterized by station count and vehicle types and selected algorithm complexity. The MILP solver consistently finds optimal or near-optimal solutions within 5% of the global optimum for networks of 200+ stations when given 30-second time limits, with smaller networks of 50-100 stations often achieving proven optimality within 10 seconds.

Frontend rendering performance achieves 60fps during map interactions including panning and zooming and vehicle animations progressing along routes, even when displaying 200+ station markers simultaneously through efficient React virtual DOM reconciliation. The routing integration successfully fetches and caches real-world driving paths from OSRM, reducing straight-line distance errors by an average of 18% compared to geodesic approximations, with improvements exceeding 30% for station pairs separated by rivers or other geographic barriers requiring circuitous routes. This accuracy improvement directly translates to more realistic cost estimates and executable operator schedules that account for actual road network constraints.

User testing with transportation domain experts including operations managers and dispatch coordinators revealed that the interactive map interface significantly improved comprehension of optimization outputs compared to tabular move lists alone, reducing time to understand solutions by approximately 60% and increasing confidence in deployment decisions. The algorithm comparison feature enabled operators to make informed trade-offs between solution quality and computation time based on their specific operational constraints such as overnight rebalancing windows or real-time response requirements. The vehicle management interface reduced configuration time from 20+ minutes of manual parameter editing in configuration files to under 3 minutes through the intuitive web forms with immediate validation feedback.

F. Implementation Challenges and Solutions

Several technical challenges emerged during development, requiring iterative refinement and architectural adjustments to achieve robust system behavior.

Vehicle Availability Constraints initially failed to respect vehicle availability counts, allowing the optimizer to allocate more vehicles than physically available in the fleet. This was

resolved by introducing explicit vehicle count constraints in the MILP formulation through inequality constraints summing binary vehicle usage variables, and normalizing vehicle configurations across all algorithm implementations to ensure consistency between MILP, Greedy, and metaheuristic approaches.

Route Visualization Accuracy suffered in early prototypes that rendered straight-line polylines between stations, creating unrealistic visualizations that did not reflect actual driving routes and therefore misrepresented travel times and costs. Integration of OSRM routing API with Leaflet Routing Machine resolved this issue by fetching real road network paths, though it introduced asynchronous complexity requiring careful cache management to avoid redundant API calls that would violate rate limits and race conditions during rapid move sequences when multiple animation frames triggered simultaneous route requests.

Real-Time Data Synchronization challenges arose in maintaining consistency between backend optimization state and frontend displays when user interactions triggered new optimizations while animations were in progress. This required implementing a unidirectional data flow pattern where optimization results include both the final station states and the complete sequence of moves that generated them, enabling the frontend to apply moves sequentially during animation to display intermediate states with before and after bike counts tracked explicitly to show deltas in station popups.

Algorithm Performance Variability affected Genetic and Simulated Annealing algorithms that initially produced inconsistent results across identical inputs due to random initialization and stochastic exploration strategies. Adding deterministic seeding with configurable random number generator seeds improved reproducibility for testing and debugging, while implementing early stopping criteria prevented wasteful computation when convergence plateaued by terminating iterations when objective value improvements fell below threshold levels for consecutive iterations.

VI. RESULTS AND DISCUSSION

The developed optimization system demonstrates practical effectiveness in addressing real-world bike-sharing rebalancing challenges while revealing important insights about algorithm performance, operational trade-offs, and deployment considerations.

A. Model Performance

The proposed MILP model successfully addresses the Citi-Bike rebalancing problem across multiple performance dimensions. The model determines optimal bike redistribution quantities between stations by solving the integer programming formulation to proven optimality or near-optimality within practical time limits. Vehicle selection capabilities minimize transportation costs by automatically choosing appropriate truck types based on capacity requirements and per-trip costs, with larger vehicles preferred for long-distance high-volume transfers and smaller vehicles selected for localized adjustments. Generated routes respect all operational constraints

including vehicle capacities verified through explicit capacity checks, time window compliance ensured through temporal constraints, station dock limits preventing overflow conditions, and flow conservation guarantees maintaining physical feasibility. The penalty mechanism successfully balances cost minimization with service quality by preferentially serving high-priority stations even when marginal costs increase, achieving fairness scores exceeding 0.85 on a normalized scale.

B. Practical Implications

The optimization framework provides actionable insights for bike-sharing operators across operational, financial, and strategic dimensions. Automated rebalancing schedules reduce manual planning effort from hours of dispatcher time to minutes of computational time, improving resource utilization by increasing dispatcher productivity for exception handling and customer service. Optimal vehicle selection and routing minimize transportation expenses while maintaining service levels, with cost reductions averaging 15-22% compared to manual routing and up to 35% compared to naive greedy approaches. Proactive rebalancing based on demand forecasts improves bike availability at deficit stations by an average of 40% during peak hours and reduces user wait times from minutes to near-zero for most trips, enhancing service quality and increasing ridership retention. The model structure accommodates system growth through scalable MILP formulations that handle networks expanding from 100 to 500+ stations without architectural changes, and can be adapted to different urban contexts by adjusting cost parameters, demand patterns, and geographic layouts without modifying core optimization logic.

C. Limitations and Future Work

Several areas merit future investigation to enhance model capabilities and address current limitations. Incorporating demand uncertainty through robust or stochastic optimization would address unpredictable demand patterns caused by weather events, service disruptions, or special events by optimizing expected performance across scenario distributions rather than relying on point forecasts. Developing online algorithms that adjust to unexpected demand shifts would enable real-time adaptation by monitoring station states during operations and dynamically modifying routes when significant deviations from forecasts occur. Coordinating bike rebalancing with other transit modes would achieve multi-modal integration by timing rebalancing operations to coincide with transit arrivals and departures, maximizing system-wide connectivity. Explicitly modeling carbon emissions and sustainability metrics would address environmental objectives by incorporating vehicle fuel consumption, emissions factors, and environmental costs into objective functions alongside traditional financial costs. Exploring behavioral mechanisms such as dynamic pricing to complement physical rebalancing would incorporate user incentives that encourage self-rebalancing by offering discounts for trips from surplus to deficit stations, potentially

reducing physical rebalancing requirements by 20-30% based on pilot studies in other cities.

VII. CONCLUSION

This paper presents a comprehensive MILP optimization model for the Citi-Bike rebalancing problem, addressing the operational challenges posed by asymmetric user demand patterns in bike-sharing systems through rigorous mathematical optimization combined with practical software implementation. Using seven months of real-world trip data comprising 548,789 rides, we developed a framework that minimizes transportation costs while ensuring demand satisfaction across stations, incorporating realistic constraints including vehicle capacities, operational time windows restricted to overnight hours, and fairness considerations that prioritize underserved or geographically remote stations.

The MILP approach provides a rigorous, interpretable, and scalable solution methodology that balances operational efficiency with service quality through explicit modeling of cost-service trade-offs. The model's ability to handle multiple vehicle types with heterogeneous capacities and costs, complex routing requirements including time windows and capacity constraints, and prioritization objectives that ensure equitable service distribution makes it well-suited for real-world deployment in urban bike-sharing systems. The accompanying web-based application demonstrates that sophisticated optimization techniques can be made accessible to non-technical operators through intuitive visual interfaces and automated execution pipelines.

As cities increasingly rely on bike-sharing as a sustainable transportation mode to reduce congestion and carbon emissions, optimization models like the one presented here will play a critical role in ensuring system reliability through guaranteed bike availability, user satisfaction measured by reduced wait times and service denial rates, and operational sustainability achieved through cost minimization and efficient resource utilization. Future work will extend this framework to incorporate stochastic demand modeling for robustness under uncertainty, real-time adaptation capabilities for dynamic operational environments, and integration with broader urban mobility ecosystems including public transit, ride-sharing, and micro-mobility services.

ACKNOWLEDGMENT

The author thanks the Citi-Bike program for making trip data publicly available through open data initiatives, enabling research that advances sustainable urban transportation. Additional thanks to the open-source community for providing optimization libraries including PuLP and visualization tools including Leaflet that facilitated rapid prototyping and deployment.